
Diagnostic des données

Gestion de capacité & détection de déviations

Systèmes de fichiers — RootVG / DataVG (AIX & Linux)

*État des lieux du jeu de données actuel (**data.csv**, en-tête seul), de sa capacité à répondre aux cinq objectifs du projet, des champs manquants, et du schéma cible recommandé pour la collecte par playbooks Ansible / agents IA.*

Objet	Analyse d'adéquation données ↔ objectifs
Périmètre	Métadonnées FS/VG non sensibles, ≥ 500 serveurs
Sortie	Schéma de collecte + référentiel « golden image »
Date	12 juin 2026
Statut	Diagnostic — pré-implémentation

Table des matières

Synthèse	2
1 L’insight principal : c’est de l’AIX	2
2 Les deux défauts bloquants	2
2.1 size_total_gb et size_available_gb → passer en octets	2
2.2 entry_type : enum non défini mais structurant	3
3 Audit des douze colonnes existantes	3
4 Ce qui manque, par finalité	3
4.1 A. Cœur capacité	4
4.2 B. Hiérarchie VG → LV → PV (le sujet même du projet)	4
4.3 C. Identité & clés	5
4.4 D. Champs normalisés (l’étape « normaliser » n’a aujourd’hui aucune colonne de sortie)	5
4.5 E. Série temporelle & prévision (obj. 3 & 4)	6
4.6 F. Conformité & données mal placées (obj. 2 & 5 — inexprimables aujourd’hui) . . .	6
4.7 G. Sorties d’analyse (à placer dans une table séparée, pas dans la donnée brute) . . .	7
5 Le référentiel : trous à combler	7
6 Recommandations structurelles	8
7 Couverture objectif par objectif (état actuel)	8
8 Schéma cible — « header type » de ce qu’on a besoin	8
8.1 Table « faits » (un enregistrement par montage / objet de stockage)	8
8.2 Table « référentiel » (golden image — fichier séparé versionné)	8
8.3 Table « findings » (sorties d’analyse — clé record_id)	9
8.4 Exemple JSONL (un hôte, hiérarchie VG→LV→PV→FS)	9
Note de méthode	9

Synthèse

Verdict en une ligne

Le schéma actuel ne permet de répondre à **aucun** des cinq objectifs de bout en bout. On compte **deux défauts bloquants** (dont le problème des tailles en GB), **un étage entier** de la hiérarchie de stockage manquant (LV / PV / espace libre du VG), **aucun référentiel** pour le sujet réel (AIX rootvg/datavg), et **aucune dimension temporelle** pour la prévision.

Les sept points à retenir :

1. **rootvg / datavg = terminologie AIX.** Le parc contient très probablement de l'AIX, or le seul référentiel (`redhat-std.txt`) ne couvre que RHEL 8/9.
2. **Tailles en GB arrondies :** tout ce qui fait < 1 GB tombe à 0. À passer en octets.
3. **Aucune taille « used »** ni espace libre du VG : la croissance n'est pas calculable.
4. **Étage LV/PV absent :** on saute `vg_name` → `path`.
5. **Snapshot unique :** pas de série temporelle ⇒ pas de prévision.
6. **Pas de champs normalisés** (classification root/data, rôle de montage).
7. **Signaux conformité & données mal placées absents** (options de montage, agrégats).

1 L'insight principal : c'est de l'AIX

`rootvg` et `datavg` sont de la terminologie LVM **AIX** (groupes de volumes, JFS2, commandes `lsvg / lslv / lspv`). Le parc mélange donc très probablement **AIX et Linux**. Or le seul modèle de référence fourni (`redhat-std.txt`) ne couvre que **RHEL 8 et 9**.

Conséquence

Pour la majorité des hôtes, l'étape « comparer au modèle de référence » n'aura **rien à quoi comparer**. Le système conclura « conforme » à tort (faux négatifs silencieux). C'est le plus gros trou, devant même le problème de taille, car il touche l'objet central du projet.

2 Les deux défauts bloquants

2.1 `size_total_gb` et `size_available_gb` → passer en octets

Le problème repéré sur `size_available_gb` touche **aussi** `size_total_gb`. Trois défauts cumulés :

Arrondi au GB entier	<code>/boot</code> (500 MB–1 GB), <code>/boot/efi</code> (500–600 MB), <code>hd5 AIX</code> (~24 MB), ou tout FS presque plein → tombent à 0 . Impossible de valider les standards sub-GB ni de détecter un petit FS plein.
GB (10^9) vs GiB (2^{30})	Les OS rapportent en base-2, un label « gb » suggère la base-10 → erreur systématique d'environ 7 % sur tous les seuils.
Pas de <code>size_used</code>	On n'a que <code>total + available</code> . <code>used = total - available</code> est faux dès qu'il y a des blocs réservés (<code>ext4</code> réserve ~5 % à <code>root</code>).

Correctif :

- `size_total_bytes`, `size_available_bytes` et `size_used_bytes` — tous en BIGINT octets, source de vérité.
- Source sans perte : Linux `df -B1` (ou `ansible_mounts[].size_total/size_available`), LVM `vgs/lvs/pvs -units b -nosuffix`, AIX `df -k × 1024` ou `PP_count × PP_size`.

- **Ne pas utiliser** la fact Ansible `ansible_lvm (size_g)` : elle ne fournit que des GB arrondis → elle reproduit exactement le bug. Tracer la source via `collection_method` pour rejeter les sources « lossy » à l'ingestion.
- Pour du lisible, garder un `*_gib dérivé` (base-2), jamais comme source.

2.2 entry_type : enum non défini mais structurant

`entry_type` pilote toute la logique aval (classification, calcul de capacité, comparaison), mais son ensemble de valeurs légales n'est documenté nulle part. **À verrouiller avant toute collecte** : `FILESYSTEM | SWAP/PAGING | VG | PV | LV | PSEUDO_FS | UNKNOWN`, avec le contrat « quelles colonnes sont obligatoires / NULL par type » (ex. `SWAP` → `path` NULL ; `VG` → `path/fstype` NULL, taille au niveau `VG`).

3 Audit des douze colonnes existantes

Colonne	Sév.	Problème principal	Correctif
<code>gsma_code</code>	moyenne	Format non défini (casce/charset) ; peut identifier un client (RGPD).	Format strict validé à l'ingestion ; pseudonymiser si identifiant.
<code>hostname</code>	haute	FQDN vs court non fixé, non unique entre comptes, pas de clé stable → double comptage du parc (≥ 500).	Forme canonique (FQDN lowercase) + <code>host_id</code> stable.
<code>vg_name</code>	haute	Confond le nom OS et la classification root/data ; vide sur Linux sans LVM.	Garder brut + <code>vg_class</code> dérivé.
<code>path</code>	haute	À normaliser pour joindre le référentiel ; pas de path pour swap ; peut contenir un nom client.	Doc « point de montage » + <code>path_normalized</code> + <code>mount_category</code> ; tokeniser.
<code>device</code>	basse	Très dépendant de l'OS, volatil, bruité.	Démoter en diagnostic + <code>device_kind</code> .
<code>fstype</code>	moyenne	Pas de vocabulaire normalisé ; pseudo-FS mélangés aux vrais → faux positifs.	Enum lowercase + flag <code>is_pseudo_fs</code> .
<code>entry_type</code>	haute	Enum non défini alors que toute la logique branche dessus.	Verrouiller un enum fermé avant la collecte.
<code>size_total_gb</code>	BLOQUANT	Arrondi GB ⇒ sub-GB à 0 ; GB vs GiB.	<code>size_total_bytes</code> (octets).
<code>size_available_gb</code>	BLOQUANT	Item + aucune taille « used » ; available vs free ambigu.	<code>size_available_bytes</code> + <code>size_used_bytes</code> .
<code>os_family</code>	moyenne	Texte libre ; surtout aucun référentiel hors RHEL.	Enum + router les OS sans golden image vers validation.
<code>os_version</code>	moyenne	Format hétérogène (RHEL 9.3 vs AIX 7200-05).	Garder brut + <code>os_major</code> dérivé.
<code>scan_timestamp</code>	haute	Fuseau non spécifié + snapshot unique ⇒ prévision impossible.	ISO-8601 UTC + <code>snapshot_id</code> + modèle append-only.

TABLE 1: Audit des colonnes présentes dans `data.csv`.

4 Ce qui manque, par finalité

Légende priorité : **critique** (un objectif est impossible sans ce champ), **haute**, **moyenne**, basse.
[†] = champ à ne collecter qu'en **agrégat non sensible** (compteurs / sommes d'octets par tranche, jamais de noms de fichiers, chemins, propriétaires ou horodatages par fichier).

4.1 A. Cœur capacité

Champ	Type	Prio	Obj	Finalité
<code>size_total_bytes</code>	octets (uint64)	critique	1,3,4	Capacité totale sans perte ; dénominateur de l'utilisation et plafond d'une prévision.
<code>size_used_bytes</code>	octets (uint64)	critique	2,3,4,5	Octets utilisés capturés directement (la soustraction est inexacte avec blocs réservés).
<code>size_available_bytes</code>	octets (uint64)	critique	1,3,4	Espace libre (dispo utilisateur non privilégié) sans perte.
<code>used_percent</code>	décimal 0–100	haute	3,4	Utilisation = used/total ; signal de seuil/sévérité et métrique phare des dashboards.
<code>reserved_bytes</code>	octets (nullable)	moyenne	3,4	Blocs réservés root (~5 % ext) ; donne le plafond effectif (ENOSPC).
<code>inodes_total</code>	entier (nullable)	moyenne	3	Un FS sature sur les inodes même avec des octets libres.
<code>inodes_used</code>	entier (nullable)	moyenne	2,3	Utilisation d'inodes ; aussi indicateur d'accumulation de petits fichiers.

4.2 B. Hiérarchie VG → LV → PV (le sujet même du projet)

Aujourd'hui on saute directement `vg_name` → `path`, en perdant l'étage LV **et** l'étage PV, et **sans** l'espace libre du VG. Sans cela, impossible de raisonner « RootVG / DataVG » ni de dire si un FS peut grandir.

Champ	Type	Prio	Obj	Finalité
<code>vg_size_bytes</code>	octets (uint64)	critique	1,3,4	Capacité totale du VG : plafond dur de croissance d'un LV.
<code>vg_free_bytes</code>	octets (uint64)	critique	1,3,4	Signal de croissance n°1 : extents libres → <code>lvextend</code> simple ; ~0 → escalade achat disque.
<code>vg_pv_count</code>	entier	moyenne	1,3	Nombre de PV/disques ; VG mono-PV = risque de résilience.
<code>vg_pe_size_bytes</code>	octets	moyenne	1,3	Granularité d'allocation (PE/PP) ; conversion PP → octets sans perte.
<code>vg_state</code>	enum	moyenne	1,3	État/santé du VG (active/inactive/exported/partial).
<code>lv_name</code>	chaîne	haute	1,2,4	LV support du FS, distinct de path/device ; cible exacte d'un <code>lvextend</code> .
<code>lv_size_bytes</code>	octets (uint64)	haute	1,3,4	Taille allouée du LV ; LV > FS détecte un FS non étendu.
<code>lv_copies</code>	entier	moyenne	1,3	Nombre de copies miroir ; LV critique non mirroré = déviation.
<code>pv_name</code>	chaîne	moyenne	1,3	Disque/PV support du VG (AIX <code>hdiskN</code> , Linux <code>/dev/...</code>).
<code>pv_free_bytes</code>	octets (uint64)	moyenne	3,4	Marge sur le PV : croissance possible sur disque existant vs nouvel achat.
<code>lv_type</code>	enum	basse	1,5	Type de LV/segment (jfs2/paging/boot/sysdump ; linear/striped/mirror).

Champ	Type	Prio	Obj	Finalité
<code>vg_type</code>	enum (nullable)	basse	1,3	AIX small/big/scalable ou LVM2 ; régit les limites de croissance.
<code>vg_max_lvs</code>	entier (nullable)	basse	1,3	Plafond AIX MAX LVs ; épuisement = déviation de croissance.
<code>vg_max_pps</code>	entier (nullable)	basse	1,3	Plafond AIX MAX PPs/PVs ; le VG peut ne plus grandir même avec disques.

4.3 C. Identité & clés

Champ	Type	Prio	Obj	Finalité
<code>record_id</code>	UUID/hash	critique	1,4	Clé primaire par ligne ; les rapports/recos/validations référencent une ligne précise.
<code>host_id</code>	UUID/machine-id	critique	1,4	Clé hôte stable ; évite le double comptage des hôtes renommés (cible ≥ 500).
<code>record_kind</code>	enum vg/lv/pv/fs_ mount/swap	critique	1,3	Discriminateur de la hiérarchie ; affine/remplace <code>entry_type</code> .
<code>snapshot_id</code>	chaîne/uuid	haute	3,4	Un run de collecte complet ; distingue « FS supprimé » de « non scanné ».

4.4 D. Champs normalisés (l'étape « normaliser » n'a aujourd'hui aucune colonne de sortie)

Champ	Type	Prio	Obj	Finalité
<code>vg_class</code>	enum rootvg/datavg/...	critique	1,2,3,5	Classification RootVG vs DataVG ; l'objet littéral du projet, OS-agnostique.
<code>mount_category</code>	enum system/data/...	critique	2,5	Catégorise un path (système vs data/app/user/boot) ; pivot du « mal placé ».
<code>path_normalized</code>	chaîne	haute	1,2	Path canonique ; joint au point de montage du golden image (évite faux positifs).
<code>vg_name_normalized</code>	chaîne	haute	1,2	Nom VG canonique (préfixes/suffixes compte retirés) ; alimente <code>vg_class</code> .
<code>hostname_normalized</code>	chaîne	haute	1,4	Hostname canonique malgré les conventions de nommage incohérentes (risque projet).
<code>os_major</code>	entier/court	haute	1	Version majeure dérivée ; join de standard déterministe (RHEL8 vs 9).
<code>os_distribution</code>	enum	haute	1	Distro précise (RHEL/Ubuntu/SLES/AIX) ; <code>os_family</code> trop grossier.
<code>host_role_env</code>	enum prod/test/...	haute	3,4,5	Environnement : un /var plein en prod est critique, en sandbox non.
<code>device_kind</code>	enum	basse	1	Compagnon catégoriel de <code>device</code> (LVM_LV/PARTITION/NFS/...).

Champ	Type	Prio	Obj	Finalité
architecture	enum	basse	1	x86_64-uefi/bios, ppc64le ; cadre les règles conditionnelles (/boot/efi UEFI).

4.5 E. Série temporelle & prévision (obj. 3 & 4)

Le schéma actuel ne contient **qu'un snapshot**. Une prévision a besoin de plusieurs points dans le temps (~14 points quotidiens consécutifs avant une projection fiable).

Champ	Type	Prio	Obj	Finalité
projected_days_to_full	jours (nullable)	haute	3,4	Livable phare : (plafond effectif – used)/vitesse ; pilote la priorisation.
growth_rate_bytes_per_day	octets/j signé	basse	3,4	Pente de used_bytes sur fenêtre glissante ; feature explicable, négative au nettoyage.
is_full_flag	booléen	basse	3	FS déjà au plafond (incident MAINTENANT vs prévu) ; vérité terrain pour backtest.
is_readonly_flag	booléen	basse	3	Montage RO : ne peut pas grandir ; évite une reco « étendre ce LV » absurde.

4.6 F. Conformité & données mal placées (obj. 2 & 5 — inexprimables aujourd'hui)

Champ	Type	Prio	Obj	Finalité
mount_options	chaîne (tokens)	haute	1,5	Drapeaux ro/nodev/nosuid/noexec ; conformité CIS (/tmp sans noexec, etc.).
is_separate_filesystem	booléen	haute	1,3,5	CIS exige /tmp,/var/log,/home séparés ; sinon risque DoS root.
system_partition_unexpected_usage_bytes	octets	haute	2,5	Octets hors sous-répertoires OS attendus : quantifie « données client sur partition système » sans nommer aucun fichier.
data_present_on_system_flag	booléen	haute	2,5	Rollup : contenu data/app sur une partition système ; finding phare agrégeable.
is_pseudo_fs	booléen	moyenne	1	Exclut tmpfs/proc/sysfs : évite des volumes de faux positifs.
is_network_filesystem	booléen	moyenne	1,2,5	FS distant (nfs/cifs) : placement anormal ; exclu des règles locales.
mtime_age_histogram [†]	objet (bucket→count,bytes)	moyenne	2,4	Histogramme d'âge (obsolescence) ; uniquement compteurs+octets par tranche.
large_alloc_histogram [†]	objet (band→count,bytes)	moyenne	2	Grosses allocations (cores, vieux backups, dumps) sans noms.
child_dir_usage_aggregates [†]	objet (subdir→bytes)	moyenne	2,4	du -max-depth=1 sur partitions système ; localise le volume.
last_modified_age_days [†]	entier jours	moyenne	2	Âge du contenu le plus récent (max mtime) ; FS data inchangé = candidat obsolète.

Champ	Type	Prio	Obj	Finalité
<code>world_writable_dir_count</code> [†]	entier	moyenne	5	Compte de répertoires world-writable ; durcissement CIS.
<code>data_owner_hint</code> [†]	enum	basse	2,5	Indice non identifiant data-sur-système ; largement redondant avec le flag (optionnel).

4.7 G. Sorties d'analyse (à placer dans une table séparée, pas dans la donnée brute)

Champ	Type	Prio	Obj	Finalité
<code>deviation_type</code>	enum	haute	1,2,5	Catégorie d'écart (missing/extra mount, under/oversized, data_on_system, no_reference...).
<code>severity</code>	enum	haute	1,3	Sévérité exigée par le livrable « rapports avec sévérité ».
<code>confidence_score</code>	flottant 0–1	haute	1	Déclenche l'étape « valider les faibles confiances avec les équipes » (human-in-the-loop).
<code>golden_image_id</code>	chaîne	haute	1	Référence du modèle comparé ; rend la comparaison auditable, expose les « sans référence ».
<code>recommendation_text</code>	chaîne	moyenne	2,4	Reco actionnable (grandir LV, migrer data, supprimer obsolète).
<code>validation_status</code>	enum	moyenne	1	Résultat de validation équipe compte ; réinjecte confirmé/faux positif dans l'entraînement.
<code>scan_confidence</code>	0–1 / enum	moyenne	1	Complétude du collecteur (du partiel, montage sauté) ; évite des escalades de faux positifs.
<code>collection_method</code>	enum	moyenne	1,3	Source de collecte ; permet de rejeter <code>ansible_lvm</code> (GB arrondi) à l'ingestion.

5 Le référentiel : trous à combler

1. **Aucun golden image AIX** alors que `rootvg/datavg = AIX`. À écrire : `hd4=/`, `hd2=/usr`, `hd9var=/var`, `hd3=/tmp`, `hd1=/home`, `hd10opt=/opt`, `hd11admin=/admin`, `hd5=boot` (~24 MB), `hd6=paging`, `hd8=jfs2log`, + `/var/adm/ras`, LV de dump. **datavg doit être SÉPARÉ de rootvg** — c'est le signal de mauvais placement central.
2. **Aucun standard non-RHEL** : Ubuntu/Debian (ext4, pas xfs), SLES, Oracle Linux, Rocky/Alma.
3. **Points de montage manquants** dans le standard RHEL : `/usr`, `/opt`, `/var` (FHS) ; `/var/log`, `/var/tmp`, `/var/log/audit` (CIS, séparés) ; `/srv`, `/data`.
4. `/boot/efi` présent pour RHEL 8 mais **absent pour RHEL 9** (toujours requis en UEFI x86_64) — à ajouter, conditionnel à l'architecture.
5. **Le référentiel est en prose, pas exploitable par machine**. Le transformer en lignes joignables sur `os_family` + `os_major` + `expected_mount_point`, avec `min_size_bytes`/`max_size_bytes` (sans perte ; le « + » de « 10-15GB+ » = max non borné / nullable, sinon faux positifs sur `/var`, `/home`), `expected_fstype`, `classification`, `separate_filesystem_required`, `expected_mount_options`, `severity_weight`, `model_version_and_source`.

6. **Chemin « no_reference »** : router explicitement vers la validation humaine au lieu de conclure « conforme » silencieusement.

6 Recommandations structurelles

- **JSON / JSONL plutôt que CSV** (déjà la préférence des objectifs) : un hôte a N VG $\rightarrow N$ LV/PV $\rightarrow N$ FS ; un CSV plat force des lignes hétérogènes pleines de NULL et **ne peut pas** porter les agrégats imbriqués (histogrammes). Si CSV imposé $\rightarrow$ éclater en **plusieurs tables** : (1) faits VG, (2) topologie LV/PV, (3) faits FS-montage, (4) référentiel versionné, (5) findings.
- **Store de faits FS = append-only**, clé (host_id, path, entry_type, scan_timestamp), jamais d'écrasement d'un « latest ». Cadence quotidienne ; ~ 500 serveurs $\times \sim 30$ chemins/jour $\approx 5,5$ M lignes/an — trivial en Parquet/BigQuery partitionné par date.
- **Séparer les sorties d'analyse** (§ 5.G) des faits bruts immuables, sinon relancer la détection écrase l'historique.
- **Octets partout** (FS, LV, VG, PV), déprécier tout *_gb.
- **Verrouiller l'enum entry_type/record_kind** avant la collecte.
- **Passe de normalisation + tokenisation** des segments identifiants (/data/{client}, /sapmnt/{sid}) avant stockage ; jamais de chaîne server:/export NFS ni d'options de montage avec credentials.

7 Couverture objectif par objectif (état actuel)

Objectif	Faisable	Bloqué par
1. Détection d'écarts FS/VG	Non	tailles arrondies + pas de référentiel AIX/non-RHEL + vg_class/mount_category absents
2. Optimisation (obsolète/mal placé)	Non	aucun signal de placement/âge/usage agrégé
3. Fiabilité / croissance	Non	snapshot unique + pas de vg-free/inodes
4. Efficacité op. / prévision	Non	pas de série temporelle
5. Réduction de risque / conformité	Non	pas de mount_options/séparation FS-classification système

8 Schéma cible — « header type » de ce qu'on a besoin

8.1 Table « faits » (un enregistrement par montage / objet de stockage)

En-tête CSV prêt à l'emploi (à préférer en JSON/JSONL avec hiérarchie imbriquée) :

```
record_id;snapshot_id;gsma_code;host_id;hostname;hostname_normalized;host_role_env;os_family;
os_distribution;os_version;os_major;architecture;record_kind;entry_type;vg_name;
vg_name_normalized;vg_class;vg_size_bytes;vg_free_bytes;vg_pv_count;vg_pe_size_bytes;vg_state;
lv_name;lv_size_bytes;lv_type;lv_copies;pv_name;pv_free_bytes;device;device_kind;path;
path_normalized;mount_category;fstype;is_pseudo_fs;is_network_filesystem;mount_options;
is_separate_filesystem;size_total_bytes;size_used_bytes;size_available_bytes;reserved_bytes;
used_percent;inodes_total;inodes_used;system_partition_unexpected_usage_bytes;
data_present_on_system_flag;last_modified_age_days;collection_method;scan_confidence;
scan_timestamp
```

8.2 Table « référentiel » (golden image — fichier séparé versionné)

```
model_id;os_family;os_major;os_version_range;architecture_scope;expected_mount_point;expected_vg;
vg_class;classification;mandatory;separate_filesystem_required;min_size_bytes;max_size_bytes;
expected_fstype;expected_lv_name_pattern;expected_mount_options;severity_weight;
model_version_and_source
```

8.3 Table « findings » (sorties d'analyse — clé record_id)

```
finding_id;record_id;snapshot_id;golden_image_id;deviation_type;severity;confidence_score;
recommendation_text;validation_status;detected_at
```

8.4 Exemple JSONL (un hôte, hiérarchie VG→LV→PV→FS)

```
{
  "host_id": "a1b2-...",
  "hostname_normalized": "srv-prod-001",
  "os_family": "AIX",
  "os_major": "7.2",
  "snapshot_id": "run-2026-06-12",
  "scan_timestamp": "2026-06-12T02:00:00Z",
  "vgs": [
    {
      "vg_name": "rootvg",
      "vg_class": "rootvg",
      "vg_size_bytes": 68719476736,
      "vg_free_bytes": 10737418240,
      "vg_pv_count": 2,
      "vg_state": "active",
      "lvs": [
        {
          "lv_name": "hd4",
          "lv_size_bytes": 2147483648,
          "lv_type": "jfs2",
          "lv_copies": 1,
          "fs": {
            "path": "/",
            "mount_category": "system",
            "fstype": "jfs2",
            "size_total_bytes": 2147483648,
            "size_used_bytes": 1288490188,
            "size_available_bytes": 858993459,
            "used_percent": 60.0,
            "is_separate_filesystem": true,
            "system_partition_unexpected_usage_bytes": 0
          }
        }
      ]
    }
  ],
  "pvs": [
    {
      "pv_name": "hdisk0",
      "pv_free_bytes": 5368709120
    }
  ]
}
```

Note de méthode

Diagnostic produit à partir de l'en-tête de *data.csv*, de *objectives.txt* et de *redhat-std.txt*, via une analyse multi-agents croisée (couverture des objectifs, audit des champs, modèle LVM AIX/Linux, modèles de référence, prévision de capacité, conformité/sécurité) puis consolidation et vérification de la nécessité et de la sensibilité de chaque champ proposé. Les noms de champs sont des propositions à figer avec l'équipe data avant début de collecte.